Project Report

Jinuo Tao, Erdenebayar Battsengel, Tsz Hei Nevin Tsui

April 2026

1 Project Overview

This project is a distributed text analysis tool that enables users to view statistical summaries of their own text files. The input files must follow a few simple formatting rules.[1] The user inputs one or more file paths via the command line, and the results are outputted as a summary table for the user to view, and the program will report a failure gracefully for any file that cannot be opened or does not exist. The user does not interact with the program after launch; they provide their file paths upfront and the results are printed to the terminal once all analysis is complete.

This project falls under the worker pool category of multi-process applications. At startup the parent process spawns a fixed pool of worker nodes arranged in a logical ring, with each node specialising in one of the eight analysis tasks. The parent injects one task per node into the ring, where each message travels clockwise from node to node until it reaches the node it is addressed to. A node that receives a message not intended for it forwards it silently to its neighbour; a node that receives its assigned task opens the file, performs its computation, and reports the result back to the parent via a dedicated status pipe. Concurrency is achieved because multiple nodes can be processing their respective tasks simultaneously — no node waits for another to finish before beginning its own work. Once the parent has collected all results it prints the summary table, sends a shutdown message around the ring, and waits for all child processes to exit cleanly.

2 Build Instructions

The project executable can be built simply by running the included Makefile via *make* (no arguments needed), generating an executable named *a3*. The actual text file(s) to be processed can then be passed in via the command line as a path (i.e. *./a3 textfolder/foo.txt textfolder/bar.txt*).

[1] Please see the appendix for more information.

3 Architecture Diagram

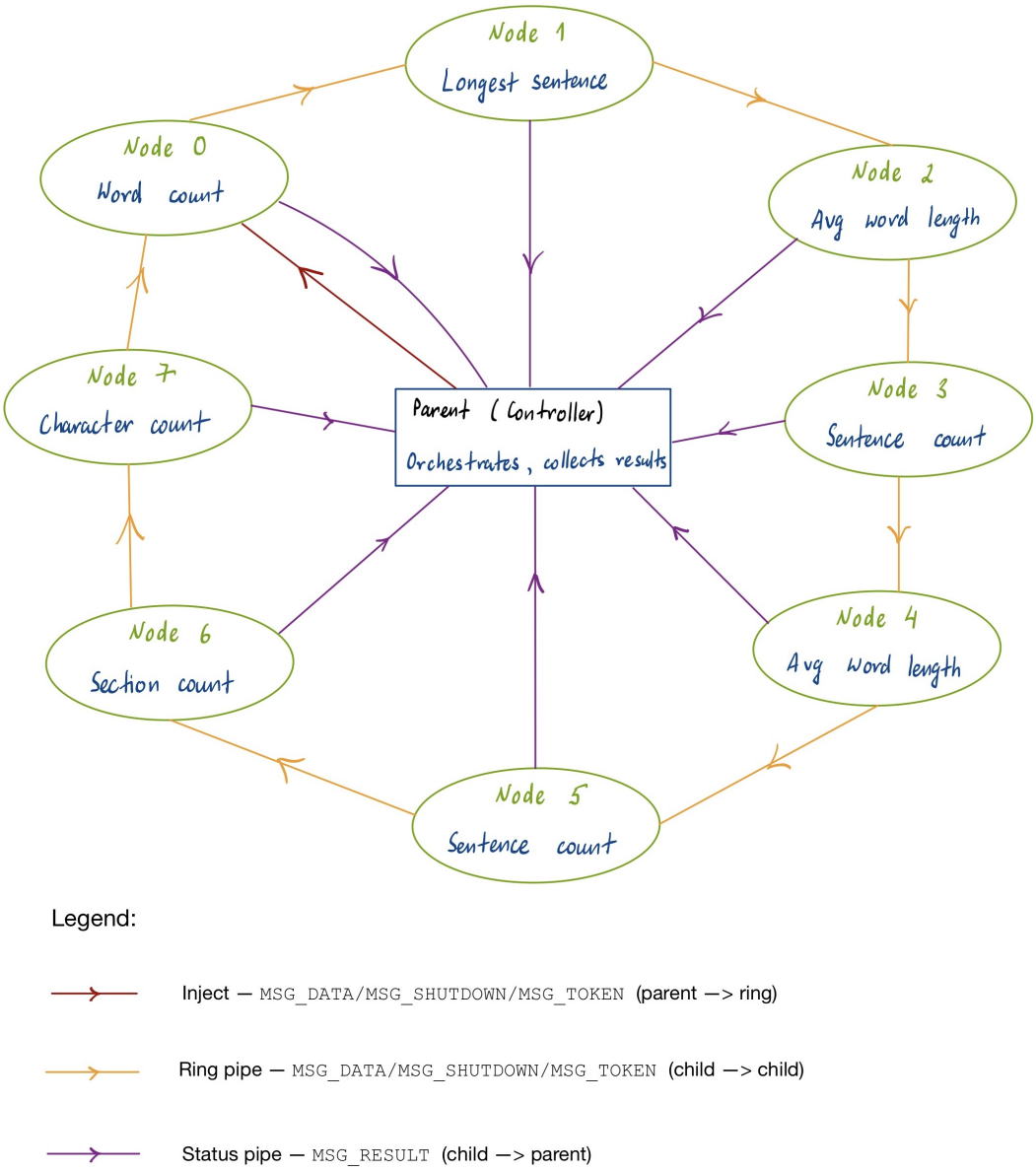

Figure 1: Architecture diagram

4 Communication Protocol

All communications between nodes (parent-to-child, child-to-child, as well as child-to-parent) are encoded with the `RingMessage` struct, which stores the following information. All members are stored as integers unless otherwise specified:

- The `type` of a message. This member is of the custom `MessageType` datatype, whose value could be `MSG_DATA` for data (task) messages and `MSG_SHUTDOWN` for shutdown messages.
- The creator and recipient of the message (`sender_id` and `receiver_id`). The value of this member is the recipient node's ID, or alternatively -1 if the message is meant for all nodes.
- The unique `task_id` of the task.
- The order in which the parent sent this task (`sequence_num`).
- The number of nodes that this message passed through (`hop_count`).
- The `status` of the task, encoded as a `TaskStatus`-type member: `STATUS_PENDING`, `STATUS_OK`, or `STATUS_ERROR`.
- The path to a file that we wish to analyze (`payload`). This is stored as a string (`char []`) of maximum length `MAX_PAYLOAD` (currently 256).
- The `result` of the task, also stored as a string of maximum length `MAX_PAYLOAD`.

We have two types of messages: ring messages and stat messages.

4.1 Ring Messages

Ring messages are sent from the parent to either a specific node, or to all nodes. A message can either request that the recipient node perform the specific task corresponding with that node (`MSG_DATA`) or shut itself down (`MSG_SHUTDOWN`). The task messages are sent to a specific node, whereas the shutdown message is sent to all nodes (so its `type` attribute is -1).

Every task-related message has a specific node as a recipient, encoded in `receiver_id`. At the start of the program, the parent injects the tasks to node zero, which then passes the other task messages to the next node in the ring via the *ring pipe*; by that time, the children would already be running a function called `run_node` that handles the receipt and forwarding of messages.

A node relay messages not meant for itself to the next node in the ring via the ring pipe, and once the message finds the recipient node, the message is processed by that node. This is done by opening the file inside the `task` function of jobs.c and performing the proper task via the helper function corresponding to the node's ID. The `task` function then sends the message back to `run_node` and closes the file. Finally, `run_node` writes the result to the parent as a *stat message* via the *stat pipe* (please see below).

There is no need to encode the specific task inside the `RingMessage` struct because the task corresponding to every `node_id` is deterministic (so node zero always handles word counts) and could be retrieved via jobs.c as opposed to being stored in the message itself.

Once our program has outputted all information, the parent sends a shutdown message, receivable by all nodes, via the `make_shutdown_msg` function. This message instructs every node to first pass the shutdown message to the next node in the ring before shutting itself down gracefully.

Error handling primarily occurs when the `write` calls fail, perhaps if a node cannot write (forward) a message to the next node in the ring pipe. In these cases where `write` fails, we print a message,[2] before using `exit` to kill the node.

[2]Sample wording: *Node 0 failed to pass the task message.*

4.2 Stat Messages

These messages are always sent from a child to the parent via the *stat pipe* (separate from the ring pipe used for the task and shutdown messages); in particular, although such messages have a `receiver_id` of -1, implying that it is receivable by all nodes, the separate nature of the stat pipe means that only the parent will ever receive a stat pipe message.

Stat messages are written by a child node upon jobs.c's `task` function returning some form of output. Then, upon receiving such a message, the parent (inside parent.c) will first print the result of the task (e.g. *Parent got result for task 1 from node 0: 11*), before storing the result (and some additional information) inside an array of `TaskEntry` struct objects (the array is called `tasks`).

The `TaskEntry` struct only helps parent.c keep track of the incoming tasks (e.g. the status and the result); this struct is *not* used for messaging purposes. At the end, the parent uses the `tasks` array to help print a summary table containing information (metrics) from all files.

Error handling for stat messages occur when such messages are written (in node.c) or read (in parent.c), via the `write` and `read` system calls respectively. Upon these system calls failing, we always print an informative message before exiting the program with some non-zero exit status.

For instance, if a child node unexpectedly exited the program, with the parent therefore reading zero bytes (due to the closure of the pipe), the parent will print a message[3] before terminating itself with the `EXIT_FAILURE` exit status.[4] The exit status is 1 if the `read` or `write` calls have themselves failed.

[3] Sample wording: *stat pipe closed unexpectedly for node 0.*

[4] Please see the "Worker Node Dying Unexpectedly" section below for more details.

5 Concurrency Model

This is a Category 1 project. The parent forks eight worker nodes at startup, and they communicate over pipes arranged in a ring. Each node is assigned one task, and task messages travel around the ring until they reach the node whose `receiver_id` matches. The synchronization is kept simple: we rely on blocking `read()` and `write()` on pipes rather than adding a more complicated concurrency mechanism.

5.1 Forking the Workers

In `main()`, we first create two pipes per node using `pipe()`: one ring pipe for messages moving from node to node, and one stat pipe for results sent back to the parent (`main.c:24-36`). We then fork all eight worker processes in a loop (`main.c:39-80`). This means the workers all exist before any task processing begins, so multiple children are alive at the same time rather than being forked one by one.

Each child closes the pipe ends it does not need (`main.c:57-75`) and keeps exactly three file descriptors:

1. the read end of its own ring pipe (`ring_pipes[i][0]`)
2. the write end of the next node's ring pipe (`ring_pipes[(i+1) % 8][1]`)
3. the write end of its own stat pipe (`stat_pipes[i][1]`)

After that, the child calls `run_node()` (`node.c:15`) and enters its main loop. Since all eight children are forked before the parent starts injecting work, at least three workers, in fact all eight, are running concurrently.

5.2 How the Parent Knows Workers Are Ready

There is no separate "ready" message. A worker is considered ready once it has entered `run_node()` and started reading from its ring pipe. At the start of the node loop, each child calls `read(ring_read_fd, &msg, sizeof(RingMessage))` (`node.c:24`). If no message has arrived yet, that `read()` blocks, so the node simply waits in the kernel until data is available.

This is the main readiness mechanism in our design. The parent does not need to poll the workers or use signals to find out whether they are idle. Instead, after all children have been forked, it starts `run_parent()` (`main.c:99`) and writes messages into the ring through `ring_pipes[0][1]`. The first message is a `MSG_TOKEN` (`parent.c:65-72`), followed by the `MSG_DATA` task messages (`parent.c:75-87`).

5.3 How Tasks Run Concurrently

The parent creates one task for each `(file, node)` pair, so every input file is processed by every node's specialized function (`parent.c:44-62`). Each task is encoded as a fixed-size `RingMessage` with a `receiver_id`, and the parent writes these messages into the ring one after another (`parent.c:75-87`).

Inside `run_node()`, each worker repeatedly reads the next message from its ring pipe. If the message is a `MSG_TOKEN`, the node increments `hop_count` and forwards it to the next node (`node.c:57-72`). If the message is a `MSG_DATA` for some other node, it also increments `hop_count` and forwards the message (`node.c:73-86`). If the message is a `MSG_DATA` for this node, then the node processes it locally (`node.c:87-117`).

The actual work is done by `task()` in `jobs.c`, which opens the file named in the message payload and dispatches to the correct helper function based on the node's task (`jobs.c:432-470`). For example, Node 0 does `word_count()`, Node 3 does `longest_word()`, and Node 7 does `character_count()`.

This allows concurrency because all workers already exist and are blocked on their own `read()` calls. Once different task messages have reached different workers, those workers can be computing at the same time. For example, one node can be running `word_count()` on one file while another node is running `longest_word()` on another file. At the same time, nodes that are not currently computing can still

keep forwarding ring messages.

Also, results do not go back through the ring. After a worker finishes a task, it creates a `MSG_RESULT` using `make_report_msg()` (`msg.c:12`) and writes that result directly to the parent through its stat pipe (`node.c:107-115`). This separates ring traffic from result traffic, so the ring is used mainly for control and task delivery while the stat pipes are used for reporting.

5.4 Collecting Children

The parent collects task results in `run_parent()` by reading from the stat pipes (`parent.c:90-123`). For each node, it expects one result per input file. These `read()` calls are also blocking, so the parent waits efficiently until a worker has actually produced a result instead of busy-waiting.

Once all expected results have been collected, the parent sends a `MSG_SHUTDOWN` into the ring using `make_shutdown_msg()` (`parent.c:163-171`, `msg.c:45`). Each worker forwards the shutdown message to the next node, closes its file descriptors, and exits (`node.c:40-56`). In this way, shutdown itself follows the same ring structure as the rest of the control messages.

Finally, after `run_parent()` returns, `main()` calls `waitpid()` on every child PID (`main.c:108-112`). This reaps all child processes and ensures that no zombies are left behind.

6 Error Handling and Robustness

6.1 Empty or Problematic Files

It is possible that the file will be empty (i.e. only contain the EOF character). Left unchecked, functions like fscanf would not function correctly, leading to unpredictable and problematic behaviour from our program.

To avoid this situation, we explicitly check for empty files before iterating our file pointer over the file with functions such as fscanf. Specifically, if we detect an empty file (whose first character would be the EOF character), we immediately return a fallback return value (0, 0.0, or an empty string) to the user without performing further read calls on the file, preventing any errors in that regard.

In addition, we enforce error checking as our file pointers traverse the documents via fscanf or fgets in the body of the task helper functions, returning a NULL result if either library function were to fail;[5] we separate these error checks from routine checks for the end of a file (via `feof`) for clarity. The child node itself, inside the `run_node` loop, will then handle the NULL result by setting the task's `status` setting to `STATUS_ERROR`.

Together, these measures ensure that program would gracefully handle all properly-formatted files.

6.2 File Does Not Exist Or Cannot Be Opened

When a worker node attempts to process its assigned task, it calls `fopen()` on the file path stored in the message payload. If `fopen()` returns `NULL` — indicating the file does not exist, the path is malformed, or the process lacks read permissions, the node sets the message status to `STATUS_ERROR`, writes a descriptive error string into the result field, and reports back to the parent via its dedicated status pipe as it would for any completed task.

The parent receives the failure report, records it in the results summary table as `FAILED`, and decrements the ACK counter accordingly. This means a bad file path is treated as a recoverable per-task failure rather than a fatal system error — the rest of the ring continues operating normally, all other nodes complete their tasks, the shutdown message still travels the ring cleanly, and all processes exit without issue.

6.3 Worker Node Dying Unexpectedly

It is possible for a worker node to terminate unexpectedly at any point during execution, for instance if it receives a fatal signal such as `SIGKILL` or encounters a segmentation fault. When this happens, the kernel automatically closes all of the dead process's file descriptors, including the write end of its status pipe. Left unchecked, the parent would block indefinitely waiting for a result that will never arrive.

To handle this, the parent's result-collection loop (`parent.c:91-126`) checks the return value of every `read()` on the status pipes. If `read()` returns 0, this indicates that the write end of the pipe has been closed, meaning the node has exited without sending its result. The parent detects this condition explicitly (`parent.c:102-106`), prints a diagnostic message identifying which node's pipe closed unexpectedly, and exits with `EXIT_FAILURE`.

We verified this by sending `SIGUSR1` to all child nodes while the program was running, simulating an unexpected crash. The parent immediately detected the closed status pipe, printed `"stat pipe closed unexpectedly for node 0"`, and exited with code 1. This confirms that the parent fails fast with a clear diagnostic when a worker disappears.

[5] Since malloc, fopen, and fclose are more integrated with the system (as opposed to the text itself), we exit the program entirely if any of the three functions were to fail.

7 Team Contributions

7.1 Erdenebayar Battsengel

Designed and implemented the pipe setup and process forking logic in main.c, creating and correctly closing all ring and status pipes across all node processes to ensure no file descriptor leaks or deadlocks. Defined the shared ring.h header including the RingMessage struct, enum types (MessageType, TaskStatus), and the message helper functions in ring.c (`make_report_msg, make_token_msg, make_shutdown_msg`). Implemented the core node loop in node.c, handling token forwarding, message routing by `receiver_id`, the shutdown sequence, and fatal error reporting to the parent via the status pipe before process exit. Drew architecture diagram and contributed to other parts of the report.

7.2 Jinuo Tao

I worked on the parent process side of the project. My main contribution was `parent.c`, where I wrote the code that builds a task list from the command-line file paths, sends `MSG_DATA` messages into the ring through the parent's write pipe to node 0, and reads back results from each node's status pipe. I created a `TaskEntry` struct to keep track of each task and used `sequence_num` to match incoming results to the right task. The parent also prints a summary table at the end and reports how many tasks succeeded or failed (ACK/NACK). I also added the `make_data_msg` function in `msg.c` and updated `make_report_msg` so it passes along `task_id` and `sequence_num`. In `main.c`, I changed it to take file paths from the command line and moved the parent logic into `run_parent`. I rewrote parts of `node.c` too, specifically, the `MSG_DATA` handling, so that a node forwards messages that aren't meant for it and only processes the ones that match its ID, then writes the result back to the parent through the status pipe. I also made the test essay files we used for testing and fixed some compiler warnings. In addition, I wrote the Concurrency Model section and the Worker Node Dying Unexpectedly subsection of the report. Lastly, I did the code walkthrough in the Demo Video, joined the video together, and uploaded it.

7.3 Nevin Tsui

I implement the vast majority of `jobs.c`. Specifically, I wrote each of the eight tasks as helper functions, converting numeric responses to strings (if necessary) within the body of those helper functions, and I wrote a function finding the right helper function (task) for each node. I also designed the logic that would open the file whose path is mentioned in the `payload`, perform the correct task on that file, and close the file afterwards. In addition, I recorded the first part of the video (i.e. the part demonstrating the program as a whole). Finally, I wrote several portions of this report, including the Communication Protocol section and the *Empty or Problematic Files* subsection of the error handling section.

8 Appendix: Full List of Functionalities and Preconditions

The program can find the following information for every file that it receives from the command line:

- Word count.
- Longest sentence (line) in the document.[6]
- Average sentence length.
- Longest word.
- Average word length, in characters.
- Sentence, section, and character counts.

The files should be formatted in accordance with the following:

- Every sentence should be a new line in the file. Namely, every sentence should end with an invisible newline character.
- Every section should end with an empty line.
- The length of every line should be lower than the `MAX_PAYLOAD` set inside the program, currently 256 characters.

[6] The first few characters of the longest sentence are printed in the summary table, but the full sentence is retrieved.